

UNIVERSITÀ
DI TORINO

DIPARTIMENTO DI
INFORMATICA

CORSO DI LAUREA
TRIENNALE

A.A. 2026/27

Canale B

Programmazione I

Teoria

Elvio Amparore

Introduzione

Strategie di apprendimento, risorse e struttura del corso per studenti di informatica.

Docenti del canale B



Prof. **Elvio Amparore**

📧 *elviogilberto.amparore@unito.it*

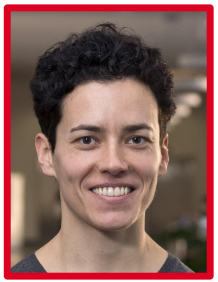
Aree di Ricerca: metodi formali, visione artificiale, intelligenza artificiale spiegabile, algoritmi distribuiti



Prof. **Valerio Basile**

📧 *valerio.basile@unito.it*

Aree di Ricerca: processamento del linguaggio naturale, rappresentazione della conoscenza, web semantico



Prof. **Elisa Marengo**

📧 *elisa.marengo@unito.it*

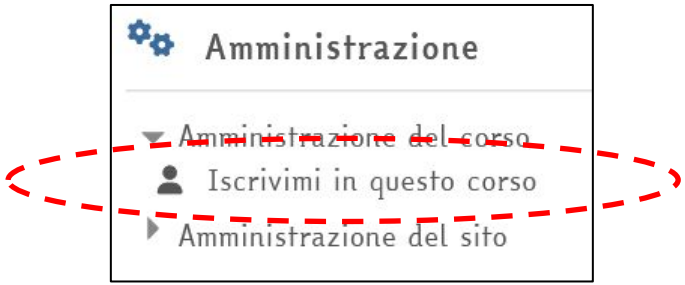
Aree di Ricerca: modellazione di processi di business, protocolli di interazione in sistemi multiagente, alfabetizzazione all'intelligenza artificiale

Organizzazione dell'insegnamento

- Secondo calendario didattico UniTo
- 48 ore di teoria in 24 lezioni da 2 ore l'una (Amparore)
  - Martedì ⏰ 11:00–13:00 in Aula B
  - Mercoledì ⏰ 09:00–11:00 in Aula B
  - Gli orari sono su **U**niversity **P**lanner in myUniTO+ o sul sito
- 30 ore di in laboratorio organizzate in 10 esercitazioni da 3 ore ciascuna a partire dal 5 Ottobre
  - Turno **T1**: matricole dispari. Mar 14-17 lab Turing (Basile)
  - Turno **T2**: matricole pari. Lun 14-17 lab Turing (Marengo)
- Appuntamenti settimanali con studenti *tutor* laureati

Pagine *Moodle* del corso

- Tutto il materiale dell'insegnamento (annunci, slides, esercizi, eventuali registrazioni) sarà condiviso attraverso questa pagina *Moodle*
- Come iscriversi:
  - Connettersi a https://informatica.i-learn.unito.it/
  - Categoria «Anno accademico 26/27»
  - Categoria «Primo anno Laurea
  - Pagina «PROGRAMMAZIONE I (B) [Cognomi E-O]»
  - Pannello «Amministrazione»



Organizzazione dell'insegnamento

- Lezioni e laboratori in presenza
- Le registrazioni delle lezioni di teoria saranno rese disponibili su Moodle dopo la lezione

ma

- La regolarità delle registrazioni non è garantita
- Le registrazioni NON sostituiscono la presenza in aula

Obiettivi del corso

- Fornire i concetti di base della programmazione imperativa strutturata di alto livello usando il linguaggio di programmazione C
- Capacità di:
  - Formalizzare la soluzione di problemi computazionali di base per mezzo di funzioni iterative e ricorsive
  - Rappresentare gli stati assunti dalla memoria durante l'esecuzione di un programma
  - Valutare la correttezza e la terminazione dei programmi per mezzo di semplici dimostrazioni formali

"*La programmazione è l'arte di tradurre i problemi in soluzioni comprensibili dal computer.*" – Edsger W. Dijkstra

Organizzazione dell'insegnamento

- *Studenti che non hanno mai programmato*: devono acquisire da zero un *modus mentis* nel trasformare problemi posti in linguaggio naturale in programmi codificati in linguaggio C. Analogamente all'apprendimento di una lingua, apprendere a programmare richiede tempo, studio ed esercizio
- *Studenti che hanno già programmato*: anche se partiamo dalle basi, seguiremo un percorso più ampio rispetto ad un tipico programma delle superiori. Tratteremo argomenti come ricorsione, modello di memoria, verifica formale della correttezza.

Il libro di testo

- Manuale di programmazione in linguaggio C di riferimento
- Disponibile online ad un prezzo ridotto
- Diverse copie sono disponibili in biblioteca al 1° piano
- Stesso testo adottato da Programmazione II



PART 1 (Introductory) Programming Fundamentals Quickstart

1. Introduction to Computers and C
Intro to Hardware, Software & Internet; Test-Drive Microsoft Visual Studio, Apple Xcode, GNU gcc & GNU gcc in Docker

2. Intro to C Programming
Input, Output, Types, Arithmetic, Decision Making, Secure C

3. Structured Program Development
Algorithm Development, Problem Solving, `if`, `if/else`, `while`, Secure C

4. Program Control
`for`, `do/while`, `switch`, `break`, `continue`, Logical Operators, Secure C

5. Functions
Custom Functions, Simulation, Random-Number Generation, Enumerations, Function Call and Return Mechanism, Recursion, Recursive Factorial, Recursive Fibonacci, Secure C

- C is one of the world's most popular and senior programming languages
- C18/C11 standards
- Topical, innovative presentation
- Rich coverage of fundamentals
- Problem-solving/developing algorithms
- 20+ fun computer-science, data-science and artificial-intelligence case studies show C as it's intended to be used—some are fully implemented, some are partially implemented and some require students to do online research
- 147 complete working programs
- 350+ integrated self-check exercises with answers
- 445 end-of-chapter exercises/projects
- Use with Windows®, macOS®, Linux®
- Visual C++®, Xcode® and GNU™ gcc

PART 2 (Intermediate) Arrays, Pointers and Strings

6. Arrays
One- & Two-Dimensional Arrays, Passing Arrays to Functions, Searching, Binary Search Visualization, Sorting, Secure C

7. Pointers
Pointer operators & and *, Pass-By-Value vs. Pass-By-Reference, Array and Pointer Relationship, Secure C

8. Characters and Strings
C Standard Library String- and Character-Processing Functions, Secure C

PART 3 (Intermediate) Formatted Input/Output, Structs and File Processing

9. Formatted Input/Output
`scanf` and `printf` formatting, Secure C

10. Structures, Unions, Bit Manipulation and Enumerations
Creating Custom Types with `structs` and `unions`, Bitwise Operators, Enumeration Constants, Secure C

11. File Processing
Streams, Text and Binary Files, CSV Files, Sequential and Random-Access Files, Secure C

- Analysis of algorithms with Big O
- Enhanced security and data science coverage as per ACM/IEEE 2020 curricula recommendations
- Use free open-source libraries and tools
- Real-world examples and data
- Traditional or "flipped" classrooms
- Secure C Programming, privacy, ethics
- Case studies in systems programming and applications programming
- Think like a developer with GitHub®, open-source, StackOverflow and more

PART 4 (Advanced) Data Structures and Algorithms

12. Data Structures
Dynamic Memory Allocation, Lists, Stacks, Queues & Binary Trees, Secure C

13. Computer-Science Thinking: Sorting Algorithms and Big O
Insertion Sort, Selection Sort, Visualizing Merge Sort, Additional Algorithms including Quicksort in the Exercises

PART 5 (Advanced) Preprocessor and Other Topics

14. Preprocessor
`#include`, Conditional Compilation, Macros/Arguments, Assertions, Secure C

15. Other Topics
Variable-Length Argument Lists, Command-Line Arguments, Multiple-Source-File Programs, `extern`, `exit/atexit`, `calloc/realloc`, `goto`, Numeric Literal Suffixes, Signal Handling

Appendices

A. Operator Precedence
B. ASCII Character Set
C. Multithreading/Multicore and Other C11/C18 Topics
D. Intro to Object-Oriented Programming

Online Appendices

E. Number Systems
F–H. Using the Visual Studio, GNU gdb and Xcode Debuggers

- Emphasis on visualization
- Static code analysis tools
- Performance, multithreading, multicore
- Questions? `deitel@deitel.com`
- Updates and errata: `https://deitel.com/chtp9`

Systems Programming Case Studies

Systems Software
- Building Your Own Computer
- Building Your Own Compiler with Infix and Postfix Notation

Embedded Systems Programming
- Webots 3D Robotics Simulator

Performance: Threading/Multicore

Applications Programming Case Studies

Algorithm Development
- Counter-Controlled Iteration
- Sentinel-Controlled Iteration
- Nested Control Statements

Random-Number Simulation
- Building a Casino Game
- Card Shuffling/Dealing with Card Images
- The Tortoise and the Hare Race

Intro to Data Science
- Data Analysis: Mean, Median & Mode

Direct-Access File Processing
- Transaction-Processing System

Visualizing Searching & Sorting

Artificial Intelligence/Data Science
- Machine Learning, GNU Scientific Library, Plotting with gnuplot, CSV Files
- NLP: Who Wrote Shakespeare's Works?

Game Programming with raylib
- SpotOn and Cannon Games

Security Via Cryptography
- Secret-Key & RSA Public-Key Crypto

Visualization with raylib
- Law of Large Numbers Animation

Multimedia: Audio & Animation

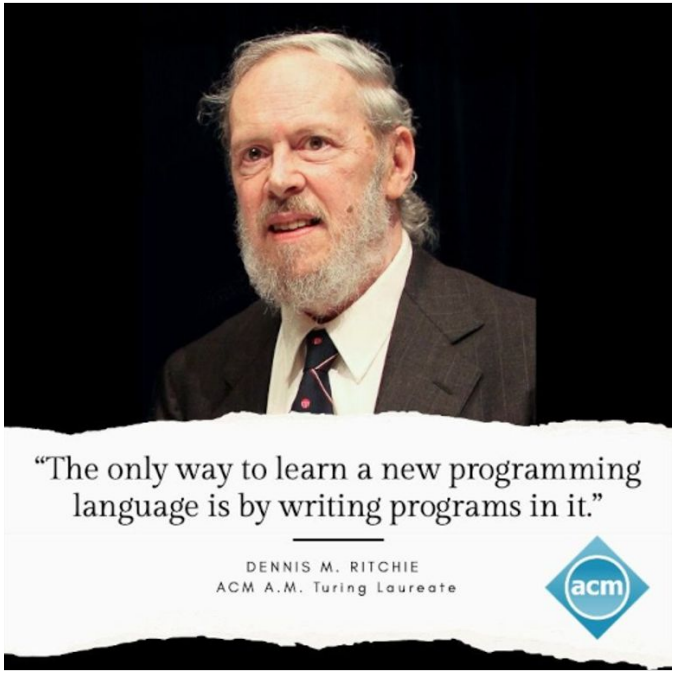
Web Services, Mashups, Cloud
- Accessing Web Services with libcurl; OpenWeatherMap JSON Results
- Rapid Applications Development with Web-Service Mashups

Tipico esame di Programmazione 1

- Esercizi in laboratorio su Moodle al PC
  - esercizi di programmazione sugli argomenti del linguaggio visti nel corso (programmazione iterativa e ricorsiva)
  - esercizi di teoria (correttezza, tipi)
  - esercizi sullo stato della memoria (esecuzione simulata dei programmi)
- Spiegheremo più dettagliatamente verso la fine, dopo aver visto gli argomenti.
- Esercizi simil-esame verranno forniti verso la fine

Programmazione ed AI generativa

- Scopo del corso: **acquisire personalmente le competenze di programmazione**
- Un LLM saprebbe risolvere gli esercizi, ma l'obiettivo non è delegare
- LLM utili per:
  - revisionare esercizi che avete già svolto
  - capire perché un programma che avete scritto non compila
- ⚠️ Usateli solo se vi aiuta, non se vi distrae dall'imparare



Come ci si prepara per un esame

- Frequentare le lezioni in presenza prendendo appunti
  - Le registrazioni servono per ripassare una lezione, non per sostituirla
- Ripassare il contenuto della lezione precedente
  - Senza la comprensione (almeno per sommi capi) della lezione precedente è difficile seguire la lezione successiva
- Approfondire sul libro di testo quanto discusso in aula
  - Alcuni argomenti non presenti (es: correttezza formale)
- Il metodo «maratona Netflix» non funziona
  - «*Professore non capisco come mai sia stato bocciato: eppure ho guardato tutte le registrazioni due volte prima dell'esame*»

Consigli di studio

- Studiare ogni giorno senza lasciare nulla indietro
- Superare un esame è molto più oneroso che studiare per una verifica
- Chiedere al docente a fine lezione e/o a ricevimento quando qualcosa non è chiaro
- Frequentare i laboratori e chiedere l'aiuto degli studenti tutor
- Sfruttare aule studio EDISU, es. piano terra PdF
- Organizzarsi in gruppi di studio con profili eterogenei
- Sfruttare supporto tutor più avanti durante l'anno quando annunciato
- Prepararsi alla «situazione esame», fuori dalla *comfort zone*
- Ogni sessione esame sarà una maturità in piccola scala

Studenti DSA o con disabilità

- Per studenti con disturbi specifici di apprendimento (DSA) certificati o disabilità temporanea o permanente, esiste una pagina dedicata: https://laurea.informatica.unito.it/do/home.pl/View?doc=studenti_dsa.html
- Gli studenti con DSA certificata devono registrarsi come tali presso l'ufficio competente per accedere a prove d'esame personalizzate (tempo extra, tutor in casi specifici)

Regole di comunicazione

- Come contattare il docente:
  - usando la casella di posta di ateneo *nome.cognome123@edu.unito.it* (no indirizzi privati su gmail, etc.)
  - con il forum di discussione nella pagina Moodle
- Come scrivere una email al docente:
  - Identificarsi (nome, cognome, anno)
  - Esporre la domanda in modo conciso e chiaro
  - Seguire la *netiquette*
- Per comunicazioni generali:
  📧 *programmazione1-docenti@di.unito.it*